

Web программирование

Курс для бакалавриата

Александр Менщиков,
к.т.н., доцент ИТМО

Введение в фронтенд. JavaScript и NodeJS

Александр Менщиков,
к.т.н., доцент ИТМО

Содержание лекции

1. JavaScript
2. Promise, Async/Await
3. AJAX
4. Введение в React
5. Node JS, npm, Vite

JavaScript

<https://learn.javascript.ru/>
<https://learnjavascript.online/>

Конец

Функции

Function Declaration

```
function sayHi() {  
    alert("Привет");  
}
```

Function Expression

```
let sayHi = function () {  
    alert("Привет");  
};
```

Function Declaration может быть вызвана раньше, чем она объявлена.

```
> console.log(sayHi1("Привет!"));  
console.log(sayHi2("Привет!"));  
  
let sayHi1 = function (text) {  
    alert(text);  
};  
  
function sayHi2(text) {  
    alert(text);  
}
```

✖ ▶ Uncaught ReferenceError: sayHi1 is not defined
at <anonymous>:1:13

Преимущество Function Expression

Можно использовать условное объявление функций.

```
let age = prompt("Сколько Вам лет?", 18);
```

```
let welcome = (age < 18) ?
```

```
    function () { alert("Привет!"); } :
```

```
    function () { alert("Здравствуйте!"); };
```

```
welcome();
```

Область видимости переменных, замыкание

Если переменная объявлена внутри блока кода { . . . }, то она видна только внутри этого блока.

```
> {  
    let message = "Hello";  
    alert(message); // Hello  
}  
alert(message); // Error: message is not defined
```

✖ ▶ Uncaught ReferenceError: message is not defined
at <anonymous>:5:7

```
> let i = 100;  
for (let i = 0; i < 3; i++) {  
    alert(i); // 0, 1, 2  
}  
alert(i); // 100
```

Тоже
замыкание

this

Ключевое слово **this** нужно для доступа к информации внутри объекта.

```
let user = {  
  name: "John",  
  age: 30,  
  sayHi() { alert(this.name) }  
};  
user.sayHi(); // John
```

```
let user = {  
  name: "user 1"  
};  
function getName() { return this.name; }  
  
alert(getName());  
  
user.getName = getName;  
alert(user.getName());
```

Стрелочные функции

```
let func = (arg1, arg2, ...argN) => expression;
```

Полезны, когда мы не хотим выходить из текущего контекста.

```
let group = {  
  title: "Our Group",  
  students: ["John", "Pete", "Alice"],  
  showList() {  
    this.students.forEach(  
      student => alert(this.title + ': ' + student)  
    );  
  }  
};  
  
group.showList();
```

Задача

```
const value = {
  checkValue: "Done",
  testArray: [
    { test: '1', check: "Done" },
    { test: '2', check: "None" }
  ],
  call: function () {
    return this.testArray.filter(function (test) {
      return test.check === this.checkValue;
    });
  }
}

console.log(value.call());
```

- Что задумано в этом коде?
- Что вы увидите в консоли?
- Как исправить, чтобы функция работала ожидаемо?

Функции генераторы. `function*`

Генераторы могут порождать (**yield**) множество значений одно за другим, по мере необходимости. Генераторы отлично работают с перебираемыми объектами и позволяют легко создавать потоки данных.

```
function* countUpToThree() {  
    yield 1;    yield 2;    yield 3;  
}  
  
const counter = countUpToThree();  
console.log(counter.next().value); // Output: 1  
console.log(counter.next().value); // Output: 2  
console.log(counter.next().value); // Output: 3  
console.log(counter.next().done);  // Output: true
```

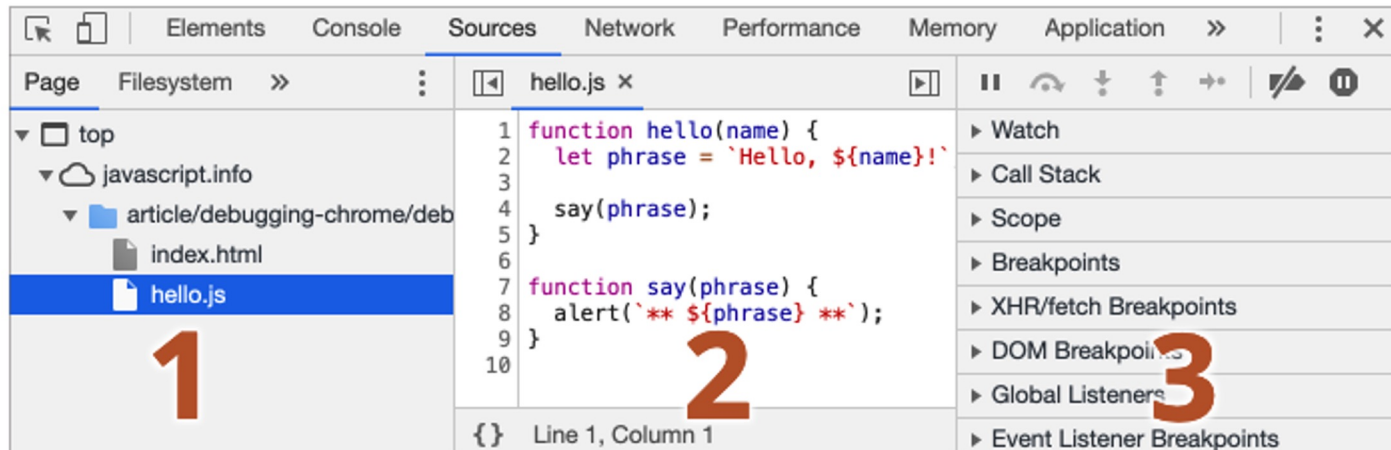
Генераторы полезны, когда нужно приостановить выполнение функции и возобновить его позже, например, при создании пользовательских итераторов или при синхронной работе с асинхронным кодом.

```
console.log("Fetching page 2...");  
yield "Page 2 data";  
  
console.log("Fetching page 3...");  
yield "Page 3 data";
```

```
[[GeneratorLocation]]: VM247:9  
▶ [[Prototype]]: Generator  
[[GeneratorState]]: "suspended"  
▶ [[GeneratorFunction]]: f* fetchData  
▶ [[GeneratorReceiver]]: Window  
▼ [[Scopes]]: Scopes[3]  
  ▶ Local (fetchData) {}
```

Дебаггер

- Исходный код
- Консоль
- Точки останова
- Команда debugger;
- Отладка



Prototype

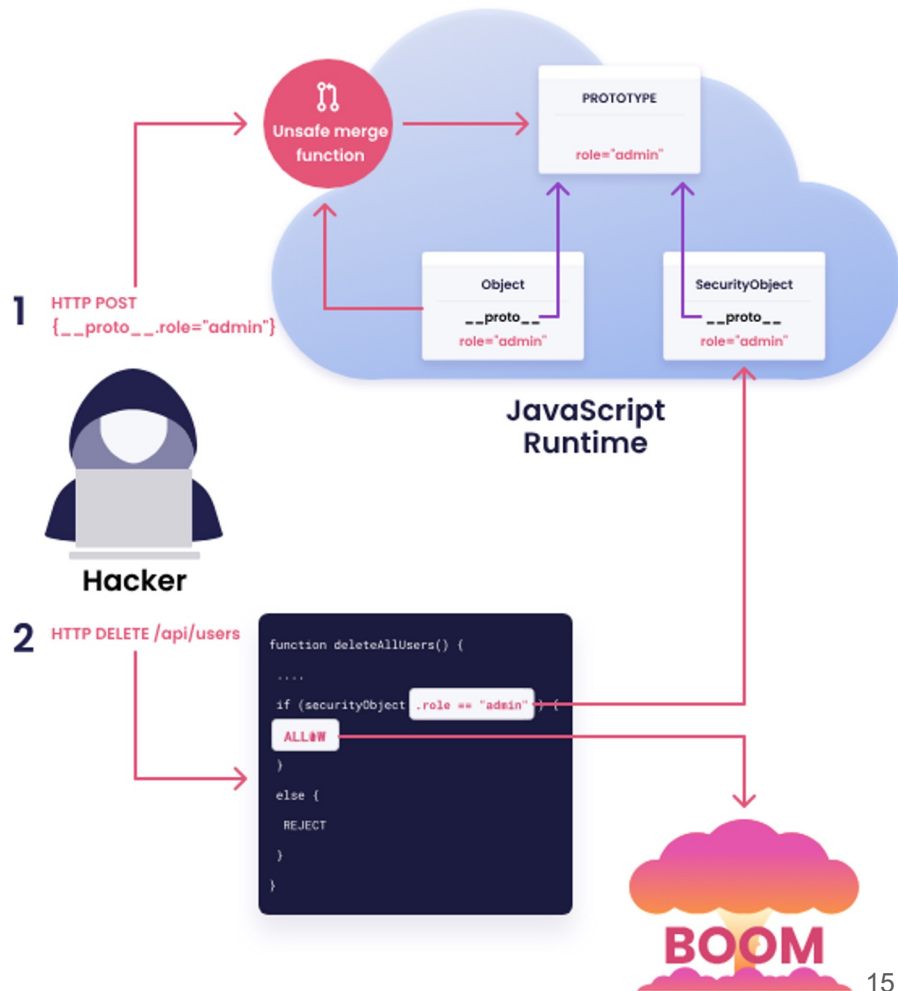
Свойство `__proto__` считается устаревшим, и по стандарту оно должно поддерживаться только браузерами.

```
function Circle() { }  
  
var shape = {},  
    circle = new Circle();  
  
// Установка прототипа объекта  
shape.__proto__ = circle;  
  
// Получение прототипа объекта  
console.log(shape.__proto__ === circle); // true
```

Prototype pollution

```
const user = { name: "Alice", role: "user" };
const params = JSON.parse(... from GET ...);
merge(user, params);
if (user.isAdmin) { do staff }
```

```
function merge(target, source) {
  for (let key in source) {
    target[key] = source[key];
  }
}
```



Prototype pollution

```
/index.html?{"__proto__":{"isAdmin:true}}
```

```
// Vulnerable merge function
```

```
function merge(target, source) {  
  for (let key in source) {  
    if (key !== "name" &&  
        key !== "role" &&  
        key !== "isAdmin") {  
      target[key] = source[key];  
    }  
  }  
}
```

```
> params
```

```
< ▼ {__proto__: {...}} ⓘ  
  ► __proto__: {isAdmin: true}  
  ► [[Prototype]]: Object
```

```
> user
```

```
< ▼ {name: 'Alice', role: 'user'} ⓘ  
  name: "Alice"  
  role: "user"  
  ▼ [[Prototype]]: Object  
    isAdmin: true  
    ► [[Prototype]]: Object
```


Promise

Callback

Колбэк-функция – это функция, переданная в другую функцию в качестве аргумента, которая затем вызывается по завершению какого-либо действия.

```
function greeting(name) {  
    alert("Hello " + name);  
}  
  
function processUserInput(callback) {  
    var name = prompt("Please enter your name.");  
    callback(name);  
}  
  
processUserInput(greeting);
```

Проблема – callback in callback

```
function loadScript(src, callback) {  
    let script = document.createElement('script'); script.src = src;  
    script.onload = () => callback(null, script);  
    script.onerror = () => callback(new Error(`Error: ${src}`));  
    document.head.append(script);  
}
```

```
loadScript('/my/script.js', function (error, script) {  
    if (error) {          // обрабатываем ошибку    } else {  
        // скрипт успешно загружен  
    }  
});
```

```
loadScript('1.js', function(error, script) {  
  if (error) {  
    handleError(error);  
  } else {  
    // ...  
    loadScript('2.js', function(error, script) {  
      if (error) {  
        handleError(error);  
      } else {  
        // ...  
        loadScript('3.js', function(error, script) {  
          if (error) {  
            handleError(error);  
          } else {  
            // ...  
          }  
        });  
      }  
    });  
  }  
});  
});
```

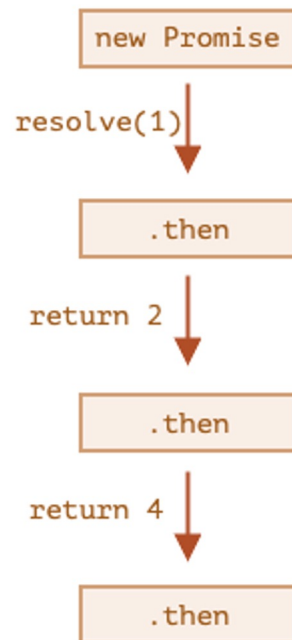
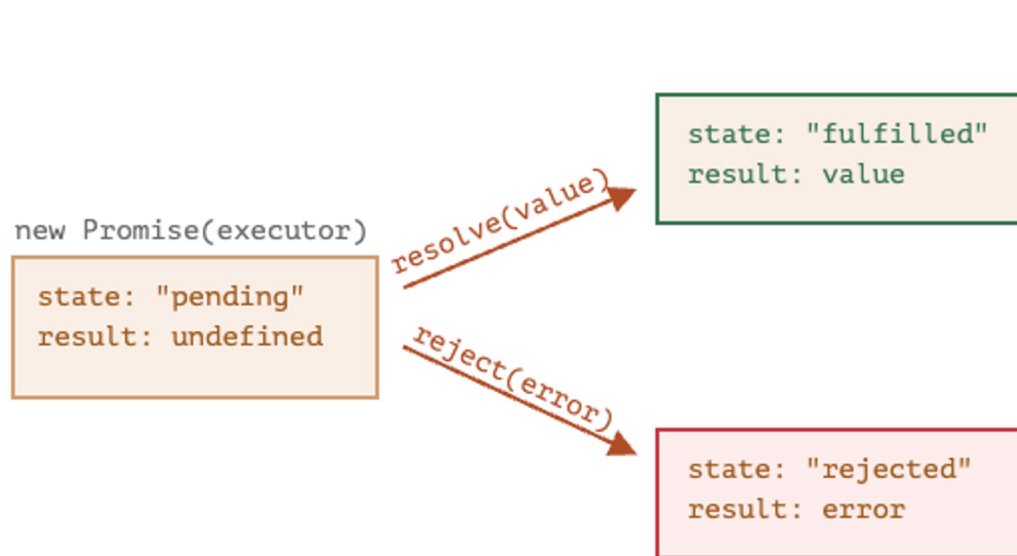


Promise

Promise – это **Object**, возвращаемый **function**, которая еще не завершила свою работу. Промис буквально представляет собой "обещание", сделанное функцией, которая через объект промиса вернет результат.

```
let promise = new Promise(function (resolve, reject) {  
    setTimeout(() => resolve("done!"), 1000);  
});  
  
// resolve запустит первую функцию, переданную в .then  
promise.then(  
    result => alert(result), // выведет "done!" через одну секунду  
    error => alert(error) // не будет запущена  
);  
  
//.catch((error) => { alert(`Error occurred: ${error}`) });
```

Цепочки промисов



Типичный пример цепочки

```
// Запрашиваем user.json
fetch('user.json')
  // Загружаем данные в формате json
  .then(response => response.json())
  // Делаем запрос к GitHub
  .then(user => fetch(`https://api.github.com/users/${user.name}`))
  // Проверяем, что пользователь найден
  .then(response => { if (!response.ok) throw new Error("Not Found"); return response; })
  // Загружаем ответ в формате json
  .then(response => response.json())
  // Показываем аватар (githubUser.avatar_url) в течение 3 секунд
  .then(githubUser => {
    let img = document.createElement('img');
    img.src = githubUser.avatar_url;
    img.className = "promise-avatar-example";
    document.body.append(img);
    setTimeout(() => img.remove(), 3000); // (*)
  })
  .catch(error => alert(error.message));
```

Запускаем Promise параллельно

```
Promise.all([  
    new Promise(resolve => setTimeout(() => resolve(1), 3000)), // 1  
    new Promise(resolve => setTimeout(() => resolve(2), 2000)), // 2  
    new Promise(resolve => setTimeout(() => resolve(3), 1000)) // 3  
]).then(alert); // когда все промисы выполнятся, результат будет 1,2,3  
// каждый промис даёт элемент массива
```


Задача – какое сообщение покажется первым?

```
let promise = Promise.resolve();
```

```
promise.then(() => alert("промис выполнен"));
```

```
alert("код выполнен");
```

Задача – какой будет порядок сообщений?

```
console.log(1);  
setTimeout(() => console.log(2), 0);  
new Promise((resolve) => resolve(console.log(3)).then((data) =>  
  console.log(data))) );  
console.log(4);
```

Event loop, микро и макро задачи

Обработчики промисов `.then/.catch/.finally` всегда асинхронны.

Приоритет:

1. Остальной код
2. Микрозадачи
3. Макрозадачи

Микрозадачи: промисы, `queueMicrotask(f)`

Макрозадачи: таймеры, обработчики кликов, AJAX

событийный
цикл



Async/await

Оператор **await** заставляет функцию, объявленную с использованием оператора **async**, ждать выполнения **Promise** и продолжать выполнение после возвращения **Promise** значения.

```
<script>
  async function f() {
    return 1;
  }
  f().then(alert); // 1
</script>
```

```
let value = await promise;
```

- await нельзя использовать в обычных функциях
- await нельзя использовать на верхнем уровне вложенности
- Ошибки можно ловить через try..catch

Пример async/await на верхнем уровне вложенности

```
<script>
  async function f() {

    try {
      let response = await fetch('http://no-such-url');
    } catch (err) {
      alert(err); // TypeError: failed to fetch
    }
  }

  f();
</script>
```

AJAX

AJAX

Асинхронный JavaScript и XML (**AJAX**) — это подход к разработке, при котором веб-приложения запрашивают данные с сервера с помощью асинхронных HTTP-запросов и обновляют только необходимые части документа, без полной перезагрузки страницы. Это может сделать страницу более отзывчивой, потому что запрашиваются только необходимые для обновления части.

Изначально для реализации **AJAX** использовался интерфейс **XMLHttpRequest**, но на данный момент для создания современных веб-приложений больше подходит **fetch() API**

XHR и fetch

XHR

- + Поддерживается даже в старых версиях IE
- Устаревший синтаксис
- Увеличение размера кодовой базы

FETCH

- + Современный синтаксис
- + Повышение читаемости кода
- + Скорость

Instead of having to write code like this

```
function reqListener() {  
    var data = JSON.parse(this.responseText);  
}  
  
function reqError(err) { ... }  
  
var oReq = new XMLHttpRequest();  
oReq.onload = reqListener;  
oReq.onerror = reqError;  
oReq.open('get', './api/some.json', true);  
oReq.send();
```

we can clean things up and write something a little more concise and readable with promises and modern syntax

```
fetch('./api/some.json')  
  .then((response) => {  
    response.json().then((data) => {  
      ...  
    });  
  })  
  .catch((err) => { ... });
```


Простой пример

```
<button onclick="fetchData()">Fetch Data</button>
```

```
<p id="fetch-output"></p>
```

```
<script>
```

```
    async function fetchData() {
```

```
        const response = await
```

```
fetch("http://127.0.0.1:8000/messages_count");
```

```
        const data = await response.json();
```

```
        document.getElementById("fetch-output").innerText = data.n;
```

```
    }
```

```
</script>
```

HTMX

motivation

- Why should only `<a>` & `<form>` be able to make HTTP requests?
- Why should only `click` & `submit` events trigger them?
- Why should only `GET` & `POST` methods be *available*?
- Why should you only be able to replace the **entire** screen?

By removing these constraints, htmx completes HTML as a *hypertext*

quick start

```
<script src="https://unpkg.com/htmx.org@2.0.3"></script>
<!-- have a button POST a click via AJAX -->
<button hx-post="/clicked" hx-swap="outerHTML">
  Click Me
</button>
```

The `hx-post` and `hx-swap` attributes on this button tell htmx:

"When a user clicks on this button, issue an AJAX request to `/clicked`, and replace the entire button with the HTML response"

HMPL

```
document.querySelector("#app").append(  
  hmpl.compile(  
    `

<button>Click!</button>  
      <div>Clicks: {{ "src": "/api/clicks", "after": "click:button" }}</div>  
    </div>`  
  )().response  
);


```

Пример, как раньше работали с AJAX и renderPartial

```
75  /**
76   * Displays a single Tasks model.
77   * @param integer $id
78   * @return mixed
79   * @throws NotFoundHttpException
80   */
81  public function actionView($id)
82  {
83      if ($this->findModel($id)->visible == 1) {
84          if (Yii::$app->request->isAjax) {
85              $view = $this->renderPartial('view_ajax', [
86                  'model' => $this->findModel($id),
87                  'accepted' => Tasks::isTaskAccepted($id, Yii::$app->user->id)
88              ]);
89              Yii::$app->response->format = Response::FORMAT_JSON;
90              return [
91                  'status' => true,
92                  'html' => $view
93              ];
94          }
95          return $this->render('view', [
96              'model' => $this->findModel($id),
97              'accepted' => Tasks::isTaskAccepted($id, Yii::$app->user->id)
98          ]);
99      } else {
100          throw new NotFoundHttpException('The task haven\' been opened yet.');
```

<https://github.com/Yozik-dev/CTF.jeopardy.checksystem/blob/master/www/modules/tasks/controllers/DefaultController.php>

https://github.com/Yozik-dev/CTF.jeopardy.checksystem/blob/master/www/modules/tasks/views/default/view_ajax.php

Введение в React

История и причины создания React

React был создан в 2011 году инженером Джорданом Уолке.

Основной причиной разработки React было желание решить проблемы, связанные с производительностью интерфейсов крупных приложений. В то время веб-приложения становились всё более сложными и тяжеловесными, что приводило к ухудшению производительности при рендеринге обновлений пользовательских интерфейсов.

Зачем и как?

- **Реактивное обновление интерфейсов** – методы обновления DOM были медленными и неэффективными для столь масштабных систем. Вместо обновления всей страницы каждый раз, когда меняются данные, React сначала обновляет виртуальную копию DOM, а затем определяет минимальный набор изменений для настоящего DOM, что значительно улучшает производительность.
- **Переиспользуемые компоненты** – в процессе создания интерфейсов разработчики хотели создать фреймворк, который позволит легко переиспользовать код для отдельных элементов страницы. Компонентный подход React позволил решать эту задачу, позволяя разделять большие интерфейсы на независимые компоненты, которые можно легко поддерживать.
- **Простота работы с состоянием** – возможность управления состоянием компонента и контроль, как изменения данных влияют на интерфейс.

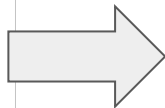
JSX – Синтаксический сахар

```
<div>
  <p></p>
  <form>
  </form>
</div>
```

HTML

```
isLoggedIn() {...}
onClick() {...}
onSubmit() {...}
```

JavaScript



```
Sidebar() {
  if (isLoggedIn()) {
    <p>Welcome</p>
  } else {
    <Form />
  }
}
```

Sidebar.js React component

```
Form() {
  onClick() {...}
  onSubmit() {...}

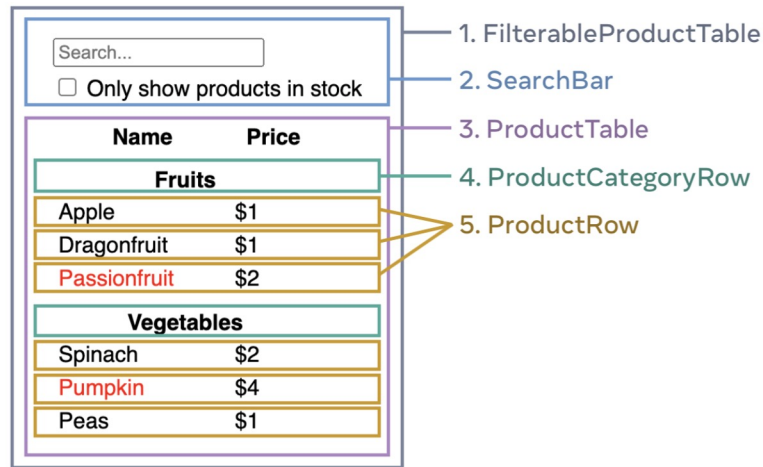
  <form onSubmit>
    <input onClick />
    <input onClick />
  </form>
}
```

Form.js React component

Философия React приложений

<https://react.dev/learn/thinking-in-react>

- Разбейте интерфейс на компоненты
- Составьте статическую версию интерфейса
- Определите минимальное представление состояния
- Определите, где хранить состояние
- Добавьте обратную связь с пользователем



Устанавливаем React

```
> npx create-next-app@latest
```

уПС

Node JS, npm, Vite

ECMAScript

ECMAScript (ES) — это стандарт, определяющий спецификацию языка программирования JavaScript.

ECMA-262 – документ, устанавливающий правила и особенности языка, выпущенный организацией ECMA International.

Основная цель ECMAScript — унифицировать и стандартизировать JavaScript, обеспечивая совместимость кода между разными браузерами и платформами.

Движки JS

Движок — это программа, которая интерпретирует и выполняет код на конкретном языке. В случае JavaScript движок отвечает за преобразование JavaScript-кода в машинный, чтобы его можно было выполнять на процессоре.

Рантайм — это более широкая среда, где код может исполняться и взаимодействовать с системой. Рантайм включает движок и дополнительные API и модули, которые помогают коду взаимодействовать с системой, сетью и другими ресурсами.

Движки:

- **V8** (by Google)
- **JavaScriptCore** (by Apple)

Рантаймы:

- **NodeJS**
- **Deno**
- **Браузер**

NodeJS

Node.js – это бесплатная кроссплатформенная среда выполнения JavaScript с открытым исходным кодом, позволяющая разработчикам создавать серверы, веб-приложения, инструменты командной строки и сценарии.

```
> node -v  
v20.12.0
```

```
import { createServer } from 'node:http';  
  
const server = createServer((req, res) => {  
  res.writeHead(200, { 'Content-Type': 'text/plain' });  
  res.end('Hello World!\n');  
});  
  
server.listen(3000, '127.0.0.1', () => {  
  console.log('Listening on 127.0.0.1:3000');  
});  
  
// run with `node server.mjs`
```

```
> npm -v  
10.5.0
```

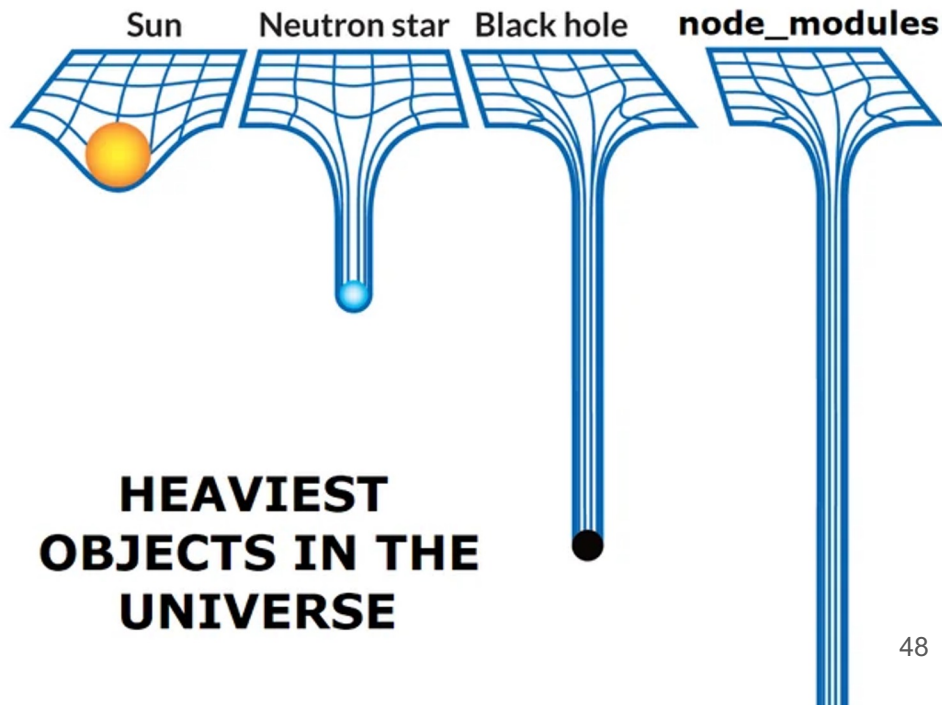
Пакетные менеджеры

- **NPM** – основной менеджер пакетов для Node.js. Устанавливается вместе с Node.js и управляет установкой, обновлением и удалением пакетов JavaScript.
- **Yarn** – альтернативный менеджер пакетов, созданный для решения проблем с производительностью и надежностью NPM.
- **PNPM** – более экономичный по пространству и производительности менеджер пакетов, оптимизированный для работы с большими проектами и монорепозиториями.
- **Bun** – новый JavaScript-рантайм и менеджер пакетов, ориентированный на высокую скорость и простоту использования.

NPM

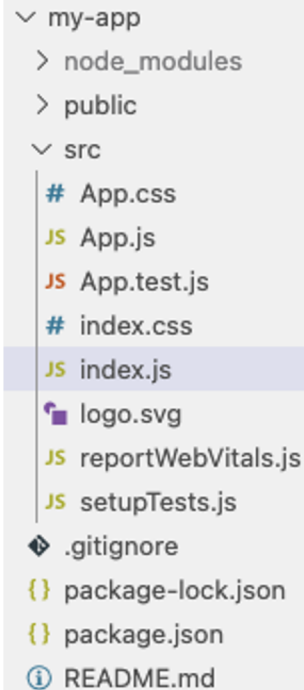
```
> npx create-react-app my-app  
> cd my-app  
> npm run start
```

Npx – Инструмент, который поставляется вместе с NPM (начиная с версии 5.2) и позволяет запускать пакеты без предварительной установки.



Изучаем фронтенд приложение

```
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import './index.css';  
import App from './App';  
import reportWebVitals from './reportWebVitals';
```



Установка Vite

#1

```
npm create vite@latest my-vanilla-js-app --  
cd my-vanilla-js-app  
npm install  
npm run dev
```



Hello Vite!

#2

```
npm create vite@latest vite-react
```

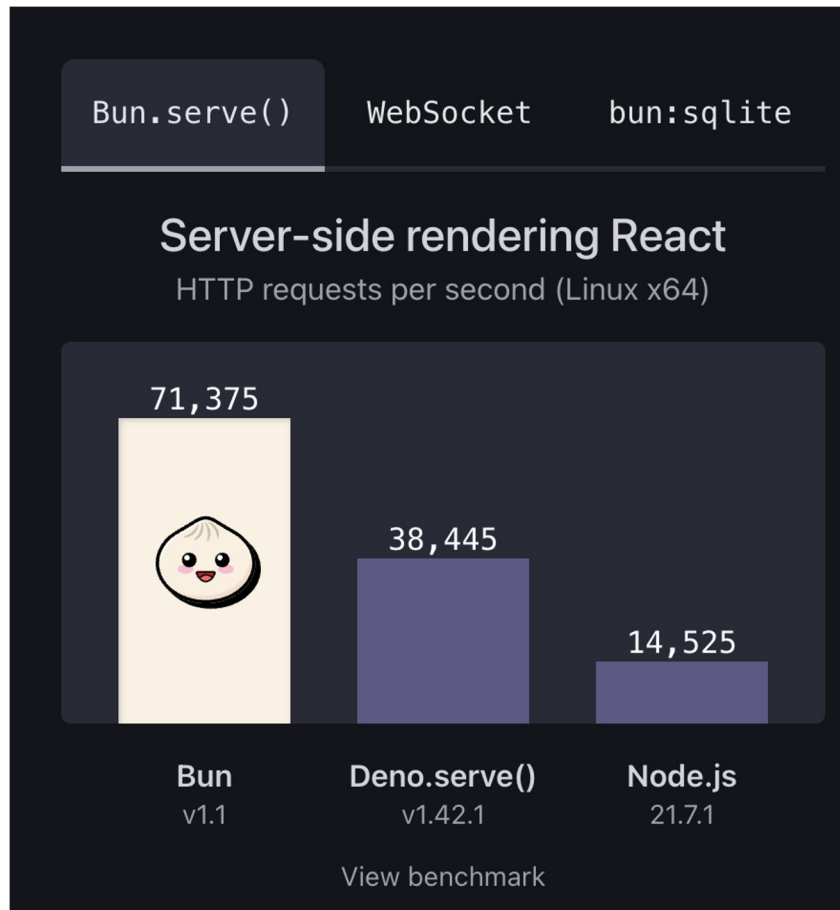
count is 0

[Click on the Vite logo to learn more](#)

Bun – очередной Runtime

npm install -> bun install

npm run dev -> bun run dev



Coding samples

https://github.com/cs-itmo/webdev_2024



Полезные ссылки

- <https://learn.javascript.ru/closure>

Вопросы?